National Philately – A Social Stamp Collection and Discovery Platform

A PROJECT REPORT

Submitted by

Santhosh Raj S (310624150040)

in partial fulfillment for the award of the Degree of

BACHELOR OF ENGINEERING

in

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled "**National Philately – A Social Stamp Collection and Discovery Platform**" is the Bonafide work of "**Santhosh Raj S (310624150040)**", who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT	**SUPERVISOR**
Dr. J. Deepa	Ms. Divyanjali

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "National Philately – A Social Stamp Collection and Discovery Platform" submitted by Santhosh Raj S (310624150040) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Design was conducted on 26.10.2026 in the Department of Computer Science and Design, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	**External Examiner**
Name :	Name :
Designation :	Designation :
Address :	Address :

ABSTRACT

National Philately is a web application that lets stamp collectors discover stamps from around the world, record what they own in a personal collection, and share their finds with other collectors. Stamp collecting is widely practised but mostly offline: a collector keeps stamps in an album, looks up values in a printed catalogue, and has no reliable record of what is owned, what is missing, or when a particular stamp was issued. Online, the information is scattered across forums, auction listings and image galleries, and the social side of collecting is largely absent.

The system is built on the MERN stack: React (Vite) for the interface, Node.js and Express for the REST API, and MongoDB with Mongoose for storage. Two roles are supported: Collector and Admin. The catalogue uses two independent axes, a category that describes the kind of issue (Historical, Commemorative, Definitive, Cultural, Sports, Science, Wildlife) and a theme that describes its subject (Independence, Space, Wildlife, Art, Architecture, Personalities, Sports, Culture). This makes a filter such as “a Sports stamp about Space” expressible. Every stamp belongs to a country and, where applicable, to a series, and carries its real image together with its source, designer and licence.

A collector can search and filter the catalogue, follow a country year by year, add stamps to a collection, see the gaps still to be filled, and open a Stamp Passport that shows progress through each country. The community area supports posts with an optional stamp attachment, likes, comments, saved posts, following, private profiles, notifications and achievement badges. The admin manages users, stamps and posts, uploads stamp images and removes inappropriate content.

The interface is responsive and mobile-first, with a compact bottom navigation on phones. Access rules are enforced on the server with JWT authentication, bcrypt password hashing, role middleware and ownership checks. The catalogue is seeded with 12 countries, 7 series and 59 real stamps, and the API is covered by an end-to-end test suite of 203 checks.

Keywords: philately, MERN stack, MongoDB, JWT authentication, digital collection, stamp catalogue, community platform, achievement system.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal, Dr. P. Deiva Sundari, and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Dr. J. Deepa, Department of Computer Science and Design, for the continuous encouragement and support.

We are extremely thankful to our Project Guide, Ms. Divyanjali, Department of Computer Science and Design, for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Design for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO. / TITLE ..... PAGE No.
ABSTRACT i
ACKNOWLEDGEMENT ii
CHAPTER 1 – INTRODUCTION 1
1.1 Overview 1
1.2 Problem Definition 1
1.3 Objective 2
1.4 Scope of the Project 2
1.5 Motivation 3
1.6 Organisation of the Report 4
CHAPTER 2 – RELATED WORK 5
2.1 Literature Review 5
2.2 Existing System 5
2.3 Proposed System 6
2.4 Comparison of Existing and Proposed Systems 7
2.5 Feasibility Study 7
CHAPTER 3 – SYSTEM ANALYSIS 8
3.1 System Requirements 8
3.2 Business Rules Enforced on the Server 9
3.3 Non-Functional Requirements 10
3.4 Actors and Use Cases 10
3.5 Software and Hardware Requirements 11
3.6 Technology Justification 12
CHAPTER 4 – SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 Modules of the System 14
4.3 Data Flow 14
4.4 Database Design 16
4.5 Class Diagram 17
4.6 Sequence Diagrams 18
4.7 API Design 19
4.8 User Interface Design 21
4.9 Security Design 22
CHAPTER 5 – IMPLEMENTATION 23
5.1 Implementation Overview 23
5.2 Development Phases 23
5.3 Key Functions 24

5.4 Deployment 28
5.5 Summary 28
CHAPTER 6 – TESTING 29
6.1 Testing Approach 29
6.2 Test Environment 29
6.3 Test Cases 30
6.4 Functional Test Results 30
6.5 Security Testing 31
6.6 Defects Found and Corrected 32
CHAPTER 7 – CONCLUSION 33
7.1 Summary of the Work 33
7.2 Outcomes 33
7.3 Limitations 34
7.4 Future Enhancements 34
7.5 Conclusion 34
APPENDIX A – REPRESENTATIVE SOURCE CODE 35
A.1 Express application bootstrap 35
A.2 User schema 35
A.3 Stamp schema and indexes 35
A.4 Collection schema with its unique index 36
A.5 Follow schema and self-follow guard 36
A.6 Authentication and role middleware 36
A.7 Registration 37
A.8 Login with either identifier 37
A.9 Catalogue query builder 37
A.10 Optional pagination utility 37
A.11 Adding and removing a collection entry 38
A.12 Collection Gaps grouping 38
A.13 Stamp Passport calculation 38
A.14 Achievement rules and idempotent unlock 39
A.15 Privacy filter for the community feed 39
A.16 Admin account toggle and cascade delegation 39
A.17 Image upload validation 40
A.18 Sample API exchange 40
SCREENSHOTS OF THE PROJECT 41
REFERENCES 43

CHAPTER 1
INTRODUCTION

1.1 Overview

Philately is the study and collection of postage stamps. It began soon after the first adhesive stamp was issued in 1840 and has grown into a worldwide hobby with national societies, specialist catalogues and international exhibitions. A stamp is a small official document: its design, denomination and date of issue record a moment in the history of the country that printed it. Collectors value stamps for their history and their artwork.

For most of its history the hobby has been paper-based. A collector keeps stamps in an album, writes the catalogue number beside each one, and looks up prices in a printed catalogue that is revised every few years. Discoveries happen at stamp fairs, in club meetings and through dealers. Nothing in this routine is recorded in a form that can be searched, counted or shared, and the knowledge built up over years lives only in the collector's memory and handwriting.

National Philately moves the hobby online. The catalogue is stored in MongoDB, so a stamp can be found by name, country, year, category, theme or series instead of by turning pages. A personal collection records which stamps the collector owns, and two derived views answer the questions a collector actually asks: what is missing, and how complete is each country. A community area lets collectors post a find, comment on it and follow the people whose collections they admire. Achievements and notifications give the hobby the small rewards that a paper album cannot.

The platform is written as a full stack web application. The client is a React single-page application built with Vite, the server is a REST API written with Node.js and Express, and the data lives in MongoDB through Mongoose. The design keeps the three tiers separate so that each can be developed, tested and deployed independently.

1.2 Problem Definition

The problems this project addresses are:

- A paper album has no backup and no search. A collector who owns several hundred stamps cannot quickly answer whether a particular stamp is already in the collection.
- Printed catalogues are expensive, quickly out of date, and organised by one axis at a time. They cannot easily answer a combined question such as "a Sports stamp from Japan about Space".

- Catalogue information online is scattered across marketplaces, forums and image galleries, so a collector keeps several browser tabs open and still cannot tell which entries are complete or trustworthy.
- Collecting is a social hobby, but existing tools offer no way to share a find, follow another collector or discuss a stamp in one place.
- A new collector has no sense of progress. Without a record of what is missing, the next purchase is a guess rather than a plan.

These problems reinforce one another. Without a searchable record, a collector buys duplicates; without a view of what is missing, a collection grows unevenly; without a community, the knowledge needed to judge a stamp stays with a few experts.

1.3 Objective

The main objective is to build a secure, responsive web platform that makes stamp collecting searchable, recordable and social. The specific objectives are:

- To let collectors register with a name, a unique username and an email address, and to sign in securely.
- To provide a catalogue that can be searched and filtered on several independent fields at the same time.
- To classify every stamp on two axes, a category that describes the kind of issue and a theme that describes its subject.
- To let a collector record the stamps they own and see the gaps that remain, grouped by country and by series.
- To show a Stamp Passport that measures progress through each country in the catalogue.
- To support a community of collectors through posts, comments, likes, saved posts and following, with a private-profile option.
- To reward progress with achievement badges and to notify collectors of activity that concerns them.
- To give an administrator tools to manage users, stamps and posts, and to upload stamp images safely.
- To enforce every access rule on the server and to keep the interface usable on a phone.

1.4 Scope of the Project

The scope of the project is a web application for discovering, collecting and discussing postage stamps.

In scope:

- Registration, login and role-based access for Collector and Admin.
- A curated catalogue of stamps with country, series, category, theme, year, denomination, story, design note and image attribution.
- Search and combined filters, country pages, a timeline and stamp detail pages.
- A personal collection with collection gaps and a per-country Stamp Passport.
- Community posts with an optional stamp attachment, likes, comments and saved posts.
- Following, private profiles, notifications, notification preferences and achievements.
- An admin console for statistics, user status, stamp management and post moderation, including safe image upload.
- A responsive, mobile-first interface.

Out of scope for this version:

- Buying and selling stamps. The catalogue records stamps; it is not a marketplace and handles no money.
- Professional valuation or grading. The catalogue does not claim to price a stamp.
- Automatic image recognition of a stamp from a photograph.
- Native mobile applications. The web application is designed to work well on phones.

The boundary is deliberate. Each out-of-scope item needs either a trusted price source, a trained model or a payment gateway, and each would change the security and legal shape of the application. The prototype keeps to what it can do correctly.

1.5 Motivation

Stamp collecting is a quiet hobby with a large following, yet the software available to a casual collector is either a marketplace built for dealers or a forum built for conversation. Neither answers the two questions that keep a collection moving: what do I have, and what should I look for next? Bringing the catalogue and the collection into one database makes those questions cheap to answer.

The project is also motivated by the social side of the hobby. A collector learns from other collectors: a story about the person on a stamp, a note about a printing variety, a photograph of a set completed after years. A small, well-moderated community around a shared catalogue can carry that knowledge without the noise of a general social network.

Finally, the project is a full stack exercise. It requires a data model with real relationships, a secured REST API, a responsive client, file handling, derived statistics and an automated test suite. Each of these is a skill the degree is meant to produce, and the hobby gives the work a concrete purpose.

1.6 Organisation of the Report

Chapter 2 reviews related work in philately, online catalogues, social collecting and web technology, and compares existing practice with the proposed system. Chapter 3 lists the functional and non-functional requirements, the business rules and the actors. Chapter 4 describes the architecture, modules, database, diagrams, API and security design. Chapter 5 explains the implementation and the key functions. Chapter 6 reports the testing carried out and the results. Chapter 7 concludes the report and lists future enhancements. The appendix contains representative source code and API examples, followed by screenshots and references.

CHAPTER 2
RELATED WORK

2.1 Literature Review

Philately and digitisation. Postal museums and national philatelic societies have spent two decades putting their holdings online. The Smithsonian National Postal Museum, for example, publishes high-resolution images of stamps with the story of the issue, and Wikimedia Commons hosts a large, freely licensed stamp collection. These projects show what a good catalogue entry contains: an image, an issuing country, a date, a denomination, a description and a credit line. They are archives rather than collecting tools, but they set the standard for the record this project stores.

Online catalogues and marketplaces. Colnect and StampWorld maintain large catalogues that let a collector mark stamps as owned, wanted or for sale, while general marketplaces such as eBay list stamps with price data. Their weakness, for this project, is that the collection is tied to a commercial account and the social layer is thin. National Philately takes the catalogue idea but keeps the collection and the community in the same application, with no trading.

Social collecting. The visual feed introduced by Pinterest and Instagram is a natural fit for stamps: each item is an image with a short caption, a creator and a set of reactions. This project applies both ideas: a post shows a stamp with its story, and achievements mark milestones such as a first stamp, ten stamps or a completed series.

Web technology for a catalogue. Token-based authentication with JSON Web Tokens (RFC 7519) and adaptive password hashing with bcrypt are standard practice for a REST API. The OWASP Top 10 guided the security choices in Chapter 5, particularly access control and injection.

Open images and attribution. A catalogue built from open images must carry its licence. Wikimedia Commons requires the source page, the author and the licence to be credited. The stamp model therefore stores source, sourceUrl, credit, licence and licenceUrl for every image, and the detail page displays them, so a collector can follow the link to the original record.

2.2 Existing System

In the existing way of working, a collector keeps stamps in an album and records them by hand. Discovery happens at fairs, in clubs, in printed catalogues and on general websites. The table below summarises the practice and its consequence.

Aspect	Existing practice	Consequence
Catalogue	Printed book or several websites	Expensive, dated and split across sources
Record of ownership	Handwritten album and a paper list	No search, no backup and easy to lose
Classification	One index at a time	A combined question cannot be asked
Discovery	Fairs, clubs, dealers and forums	Depends on place and personal contacts
Community	Club meetings and general social media	Conversation is separated from the catalogue
Progress	A rough count by eye	No reliable view of gaps or completion

Limitations of the existing system. The existing practice has five weaknesses. First, it is not searchable: a fact known to the collector cannot be retrieved quickly. Second, it is not durable: paper is easily damaged and a list kept in a notebook has no backup. Third, it is not comparable: two collectors classify the same stamp differently because there is no shared scheme. Fourth, it is not social in a useful way: a general social network has no idea what a stamp is, so a post cannot link to a catalogue entry. Fifth, it gives no sense of progress, because nothing counts what is owned against what exists.

2.3 Proposed System

The proposed system replaces these practices with a single web platform. Its main differences are:

- A shared catalogue. Every stamp is one record with a country, a series, a category, a theme, a year and a story, so two collectors describe it the same way.
- A searchable collection. One row per owned stamp, protected by a unique index, so the collection is a real record rather than a list.
- Two-axis classification. Category and theme are queried independently, which makes precise filters possible.
- Derived progress. Collection Gaps and the Stamp Passport are computed from the live catalogue on each request, so they cannot become stale.
- A community attached to the catalogue. A post can reference a stamp, and a comment sits under the post rather than in a separate forum.
- A rewarded, moderated space. Achievements mark milestones, notifications respect preferences and the admin can remove content.

2.4 Comparison of Existing and Proposed Systems

Feature	Existing practice	Proposed system
Record of collection	Paper album and a handwritten list	Collection rows in MongoDB with a unique index
Catalogue	Printed book or unrelated websites	One searchable catalogue with countries and series
Classification	One index at a time	Category and theme queried independently
Progress	Counted by eye	Collection Gaps and a per-country Stamp Passport
Community	Club meetings and general social media	Posts, comments, likes, saved posts and following
Recognition	None	Ten achievement badges and in-app notifications
Moderation	Club committee	Admin console for users, stamps and posts

2.5 Feasibility Study

Technical feasibility. Every component uses mature, free and well-documented technology. The MERN stack is widely taught and supported, MongoDB offers a local server and a free Atlas tier suitable for a prototype, and the React ecosystem supplies routing, HTTP and styling without infrastructure. The image catalogue is served from disk through an Express route, which avoids a separate file service.

Economic feasibility. The prototype runs on free software and can be demonstrated on a laptop with a local MongoDB instance. Hosting static client files and a small Node service is inexpensive, and the images are already in the repository. There is no payment gateway, so the recurring cost is hosting.

Operational feasibility. The target users are collectors of all ages, so the interface uses clear labels, a search bar that is always visible and a compact bottom navigation. The landing page explains the three actions, discover, collect and connect, and the catalogue can be browsed without an account.

Legal feasibility. Stamp images come from Wikimedia Commons under open licences, and each entry records the source, author and licence. The application does not sell or value stamps, so it needs no dealer’s licence.

CHAPTER 3
SYSTEM ANALYSIS

3.1 System Requirements

Requirements were collected by studying how a collection is kept today, by reading the documentation of existing catalogues, and by listing what each role needs to do. They are grouped into functional requirements, business rules and non-functional requirements.

Functional requirements

ID	Requirement	Role
FR1	Register with a name, a unique username, an email address and a password of at least six characters	Visitor
FR2	Log in with an email address or a username and receive a JWT	All
FR3	Search the catalogue by free text across name, depicted person, theme, story and country	All
FR4	Filter by country, exact year, decade, year range, category, theme and series, and sort the result	All
FR5	Open a stamp detail page with its image, story, design note, depicted person and attribution	All
FR6	Browse countries with flag, ISO codes, region, stamp count and a year-by-year timeline	All
FR7	Browse the catalogue as a timeline grouped by era and year	All
FR8	Add stamps to a personal collection and remove them again	Collector
FR9	See Collection Gaps, grouped by country and by series, with completion percentages	Collector
FR10	See a Stamp Passport with per-country progress, category coverage and date span	Collector
FR11	Create a post with text and an optional stamp attachment	Collector
FR12	Like, comment on and save posts	Collector
FR13	Follow and unfollow collectors and view follower and following lists	Collector
FR14	Mark a profile private and edit the profile and interests	Collector
FR15	Receive notifications, mark them read and choose which events notify	Collector

ID	Requirement	Role
FR16	Earn achievement badges for collection and community milestones	Collector
FR17	View statistics and manage users, stamps and posts, including safe base64 image upload	Admin

Explanation of the main requirements. Registration (FR1) collects only what is needed to identify a collector. The username and the email address both carry a unique index, which is what makes duplicate registration impossible even if two requests arrive together. Search (FR3) and the combined filters (FR4) are open to visitors, because a catalogue that cannot be browsed without an account is hard to evaluate.

The collection requirements (FR8 to FR10) are the heart of the application. Adding a stamp is a single row, but the two derived views are computed from the whole catalogue, so they always match the current state. The community requirements (FR11 to FR14) are restricted to signed-in collectors. Admin requirements (FR17 and FR18) are restricted to the admin role on the server, not only in the menu.

3.2 Business Rules Enforced on the Server

The following rules are enforced in the controllers and the schema, never only in the interface. A rule that is hidden by a disabled button can be bypassed by anyone who sends the request directly, so every rule is checked again on the server.

- A collector may own a stamp only once. The unique index on (user, stamp) makes a duplicate impossible even under concurrent requests.
- A follow is one row per direction, with a unique index on (follower, following). A schema guard rejects a user who tries to follow themselves.
- A private profile is readable only by its owner and by collectors who follow it. The rule is applied to the profile, its collection, its posts, the comments on those posts and the public community feed.
- An achievement is granted once. The unique index on (user, achievement) makes the unlock idempotent, and the controller handles a duplicate-key race without aborting the remaining unlocks.
- A notification is created only when the matching preference is on. Turning a preference off stops the message, not only its display.
- An admin cannot deactivate another admin. The user toggle refuses the action, so the platform cannot lose its last administrator by accident.

- Deleting a stamp removes every collection row that pointed at it, and deleting a post removes its comments and notifications. Without the cascade those rows become orphans that quietly break the counts.
- An uploaded image must be PNG, JPEG, WebP or GIF, must pass a magic-byte check and must be no larger than 4 MB. The file extension supplied by the client is not trusted.

3.3 Non-Functional Requirements

Category	Requirement
Usability	Mobile-first layout, clear labels, a visible search bar, large cards and a compact bottom navigation on phones
Performance	Indexed queries, optional pagination through ?page and ?limit, and catalogue responses that keep the body a plain array
Reliability	Derived counts and unique indexes prevent drift; missing references are filtered out instead of being shown as empty rows
Security	Hashed passwords, JWT authentication, role and ownership checks, login throttling, upload type checks and no password in any response
Maintainability	One taxonomy file, one controller per resource, shared pagination helpers and a single error handler
Portability	Only standard Node and browser APIs; the image folder can be moved behind a storage interface
Accessibility	High contrast, keyboard-reachable controls, alternative text for stamp images and a layout that works at 360 px

Discussion. Usability is treated as a requirement and not a decoration. A collector browsing on a phone in a stamp fair needs to search with one hand and read a card without zooming, so the catalogue keeps one primary action per screen and a compact bottom navigation. Performance is expressed in terms of the query plan: the stamps collection has indexes on category and theme, on country and year, on year, on series and on view count, and the free-text search uses escaped regular expressions so that a user cannot inject a pattern that scans the collection slowly.

3.4 Actors and Use Cases

Visitor. Browses the landing page, the catalogue, country pages, the timeline and stamp details, and searches without an account.

Collector. Registers, signs in, maintains a collection, views gaps and the passport, creates posts, comments, likes, saves, follows other collectors, manages a profile and its privacy, reads notifications and earns achievements.

Admin. Signs in with a seeded administrator account, reads platform statistics, activates or deactivates accounts, adds, edits and deletes stamps, uploads stamp images and removes posts.

Use case	Primary actor	Pre-condition	Result
Browse catalogue	Visitor	None	Matching stamps are listed
Search and filter	Visitor	Valid filter values	Filtered result, or 400 on an invalid category or theme
Register	Visitor	Email and username unused	Account created and signed in
Login	Collector	Account active	JWT issued and session restored
Add to collection	Collector	Signed in, stamp exists	Collection row created; achievements checked
View gaps	Collector	Signed in	Missing stamps by country and series
Set private	Collector	Signed in	Collection and posts hidden from non-followers
Upload image	Admin	Admin token, valid image	File stored and URL returned
Delete post	Admin	Post exists	Post, comments and notifications removed

3.5 Software and Hardware Requirements

Software requirements

Component	Technology
Front end	React 18, Vite, React Router, Axios, Tailwind CSS
Back end	Node.js 18 or later, Express.js 4
Database	MongoDB with Mongoose 8
Authentication	JSON Web Tokens, bcryptjs
Image handling	Base64 data URL upload, magic-byte check, static serving from server/public/stamps
Testing	Node test scripts using the built-in fetch API
Development tools	Visual Studio Code, PowerShell, Git, Google Chrome
Operating system	Windows 10 or 11 for development; any modern OS for use

Hardware requirements

Item	Development machine	End-user device
Processor	Dual-core 2 GHz or better	Entry-level laptop or phone

Item	Development machine	End-user device
Memory	8 GB RAM recommended	2 GB RAM
Storage	2 GB free for tools and dependencies	Small, since the app runs in the browser
Display	1366 x 768 or better	360 px wide or wider
Network	Broadband for installing packages and seeding	Any connection; images are cached by the browser

3.6 Technology Justification

React with Vite. React's component model suits an application made of repeated pieces such as stamp cards, filter panels, post cards and comment lists. Vite gives fast start-up and hot reload during development and produces a small optimised bundle for deployment. React Router keeps the catalogue, country, stamp and community URLs shareable.

Node.js and Express. Using JavaScript on both sides lets one developer write the client and the server and share naming and validation rules. Express is minimal, so the middleware chain (authentication, role, upload, error handling) is explicit and easy to explain. The same language also keeps the test script short, because it can reuse the models and the connection code.

MongoDB with Mongoose. A stamp naturally holds nested and varying information such as attribution fields and story text, and different countries need different amounts of detail. A document database stores such records without rigid joins, while Mongoose adds schemas, validation, references and indexes so that the data stays consistent. The catalogue is also read mostly as whole documents, which suits this model.

JWT and bcryptjs. A stateless token allows the server to be restarted or scaled without a session store, and it carries only the user id. bcrypt is deliberately slow and salted, so a stolen database of password hashes cannot be cracked quickly.

Tailwind CSS. The utility classes keep the styling beside the markup, so a responsive layout is quick to adjust. The postal palette (paper, postal red, deep blue and cream) is defined once in the Tailwind configuration and used across cards, buttons and headings.

Repository scripts. The catalogue is curated, not scraped at run time. A manifest pins the exact Commons file for each stamp, a fetch script downloads the images and writes the seed file, and a preview script produces a contact sheet. The data is therefore reproducible.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

National Philately follows a three-tier architecture. The presentation tier is a React single-page application served as static files. The application tier is an Express REST API that holds all business rules. The data tier is a MongoDB database accessed through Mongoose models, together with a folder of stamp images served through a controlled static route. The client never talks to the database directly; every read and write passes through the API, where authentication, authorisation and validation are applied.

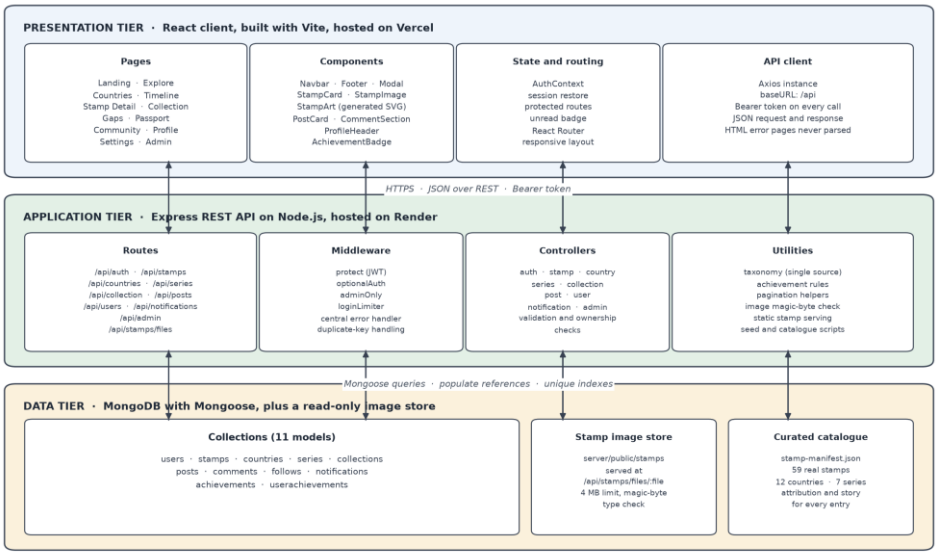


Fig. 4.1 Three-tier architecture of National Philately

The presentation tier. React renders the catalogue, the collection and the community from JSON responses. React Router maps URLs such as /stamps, /countries/japan and /community to pages, so a stamp or a country can be bookmarked and shared. An Axios instance attaches the stored token to each request and redirects to the login page when the server answers 401. Tailwind CSS supplies the responsive layout, and a small set of shared components (StampCard, FilterPanel, PostCard, EmptyState) keeps the pages consistent.

The application tier. Express receives a request, passes it through the middleware chain and hands it to a controller. The chain consists of JSON body parsing, a login limiter on the two credential routes, an authentication middleware that verifies the JWT, an optional authentication middleware for public pages that personalise their response, a role guard for admin routes and a central error handler. Controllers use Mongoose models

and return a JSON body with an explicit HTTP status; no controller writes to the database without first checking who is asking.

The data tier. MongoDB stores eleven collections. Mongoose schemas declare the fields, the enumerations, the references and the indexes, and a few schema hooks keep derived fields correct. Stamp images are stored as files under server/public/stamps and are served by a read-only route at /api/stamps/files, so the path that appears in a stamp record is independent of the host name. The database connection string and the token secret come from environment variables.

4.2 Modules of the System

The application is divided into modules along the resources it serves. Each module has a route file, a controller and one or more models, which keeps a change to one resource from spreading across the code base.

Module	Responsibility	Main files
Authentication	Registration, login, token issue and the current user	authController, authRoutes, auth middleware
Users	Public profiles, profile editing, privacy, following and search	userController, userRoutes, Follow model
Catalogue	Stamp list, detail, search, filters, facets and view count	stampController, stampRoutes, Stamp model
Countries and series	Country pages, regions, flags and per-country series	countryController, seriesController, Country and Series models
Collection	Owned stamps, gaps, passport and collection statistics	collectionController, collectionRoutes, Collection model
Community	Posts, comments, likes and saved posts	postController, commentController, Post and Comment models
Notifications	Creation, listing, read state and preferences	notificationController, Notification model
Achievements	Definitions, evaluation rules and unlocked badges	achievementController, Achievement and UserAchievement models
Administration	Statistics, user status, stamp management, post moderation and image upload	adminController, adminRoutes, upload middleware

4.3 Data Flow

A typical read starts in the browser. A page component calls a function in the API service, which sends a GET request with the token when one exists. Express authenticates

the request, the controller builds a Mongoose query from the validated query parameters, MongoDB returns the documents, and the controller maps them to plain JSON. The client stores the result in component state and renders it. No HTML is generated on the server, so the same API can later serve a mobile client.

A typical write follows the same path with three additions. First, the request body is validated before a model is touched. Second, ownership or role is checked against the resource being changed. Third, side effects such as unlocking achievements or creating a notification run after the main write and are allowed to fail without undoing it, because a missing badge is a smaller problem than a lost collection entry. The sequence below shows the browse and save flow; the next section shows the community flow.

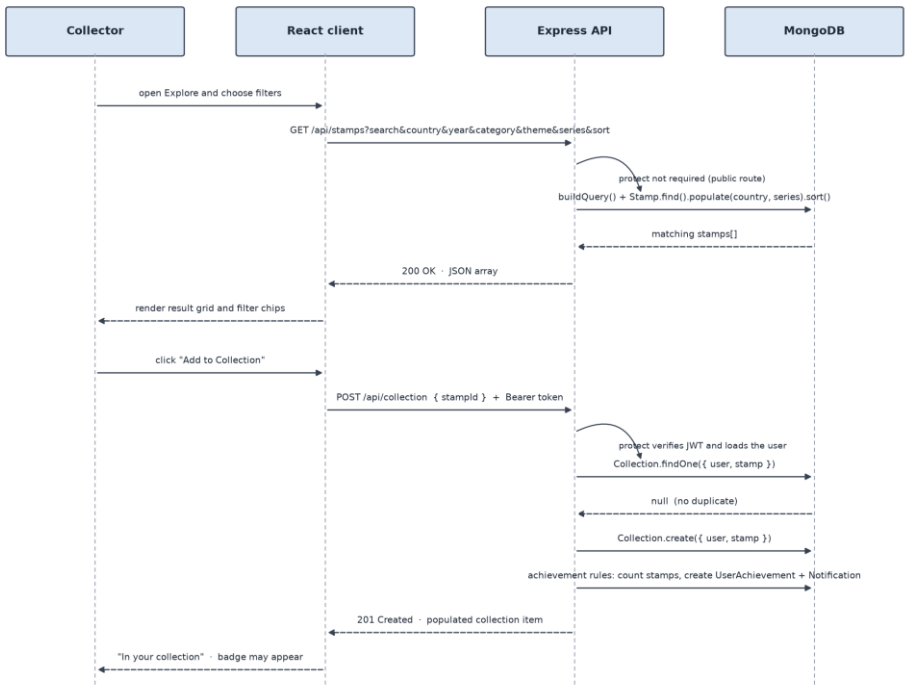


Fig. 4.2 Sequence of browsing the catalogue and saving a stamp

- The client sends GET /api/stamps with the filter, sort and pagination parameters.
- The controller validates the enumerations, escapes the search text and builds the query.
- MongoDB returns the page; the controller returns a JSON array with X-Total-Count and X-Page headers.
- The collector presses Save; the client sends POST /api/collection with the stamp id in the body and the token in the header, and the server inserts one row and evaluates the achievement rules.

4.4 Database Design

The database has eleven collections. References are stored as ObjectId values with a Mongoose ref, so a document can be populated when a page needs the related record and left as an id when it does not. The table lists the purpose of each collection and its most important fields.

Collection	Purpose	Key fields
users	Collector and administrator accounts	name, username, email, password, role, bio, interests, profileImage, isPrivate, notify* flags, isActive
stamps	Catalogue entries	name, country, countryName, series, year, category, theme, denomination, story, person, image, credit, license
countries	Issuing countries	name, iso2, iso3, region, flag, currency, firstIssueYear, description
series	Named groups of stamps	name, country, category, theme, issuedFrom, issuedTo, totalStamps
collections	Stamps owned by a collector	user, stamp, addedAt
posts	Community posts	user, content, image, stamp, likes, savedBy
comments	Replies to posts	post, user, content
follows	Directed follow edges	follower, following
notifications	In-app messages	user, type, message, link, read
achievements	Badge definitions	name, description, icon
userachievements	Badges unlocked by a collector	user, achievement, earnedAt

Relationships. A user has many collection rows, posts, comments, notifications and unlocked achievements. A stamp belongs to one country and optionally to one series, and appears in many collection rows and posts. A post belongs to one author, may reference one stamp, has many comments and stores the ids of the users who liked it and the users who saved it. A follow links two users, and a notification belongs to one recipient and carries a link to the object it describes. These relationships are shown as a class diagram in Section 4.5.

Indexes and constraints. Unique indexes on users.email and users.username make duplicate registration impossible, and a sparse unique index on users.googleId stops a Google account from being linked twice. A compound unique index on collections (user, stamp) prevents a collector from owning the same stamp twice, and a compound unique

index on follows (follower, following) prevents a duplicate follow; a schema validator rejects a self-follow. A compound unique index on userachievements (user, achievement) makes an unlock idempotent. The stamps collection carries indexes on category with theme, country with year, year, series with seriesNumber and viewCount, plus a text index used by search. Notification reads use a compound index on user and createdAt.

Referential integrity. MongoDB does not enforce foreign keys, so the application enforces them. A controller checks that a referenced stamp or user exists before writing a row. When a stamp is deleted, the controller removes every collection row that pointed at it; when a post is deleted, its comments and the related notifications are removed. The gaps and passport views ignore an owned id that no longer resolves to a stamp, so a stale row cannot inflate a percentage even if one survives a manual database edit.

4.5 Class Diagram

The class diagram shows the eight entities that carry the behaviour of the application. User, Stamp, Country and Series describe the catalogue and its owners. Collection and Follow are explicit relationship objects because each carries its own attributes or constraints. Post and Achievement complete the community and reward side. Comment, Notification and UserAchievement are supporting entities with a single clear owner.

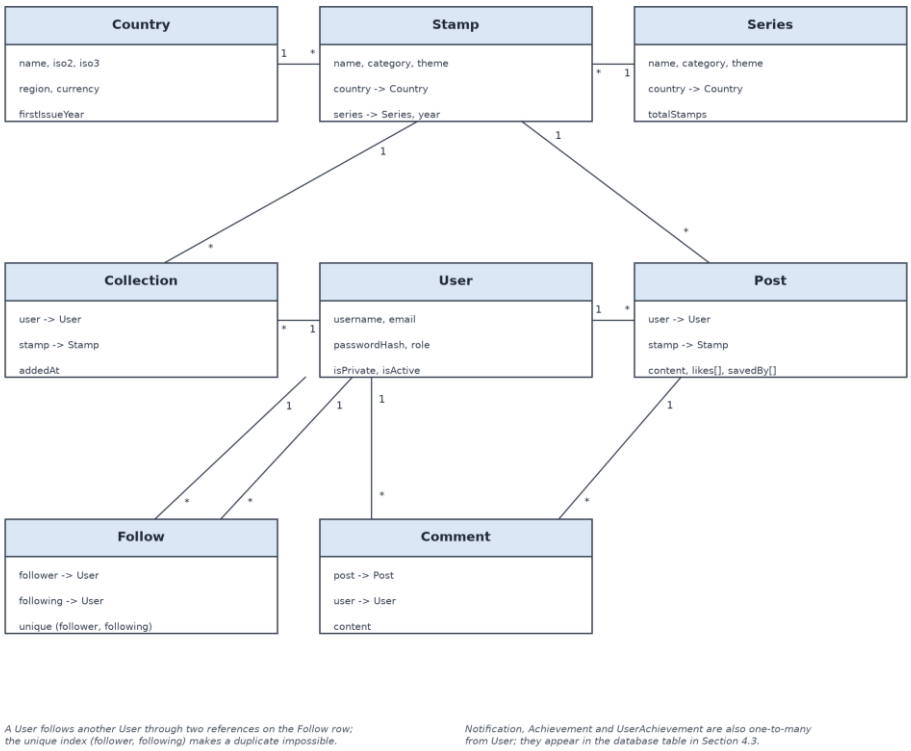


Fig. 4.3 Class diagram of the core entities

A User owns many Collection entries and many Posts, and follows other users through Follow. A Stamp belongs to a Country and optionally to a Series; a Collection entry joins a User to a Stamp. A Post is written by a User and may attach a Stamp, and it holds a list of likes. An Achievement is unlocked for a User through UserAchievement, which records the date. The multiplicities in the diagram are the multiplicities the code enforces: one country to many stamps, one user to many collection rows, and one stamp to at most one row per user.

4.6 Sequence Diagrams

The sequence diagrams show the two flows that matter most: saving a stamp, and creating a post that triggers an achievement. Both diagrams begin in the browser, pass through the API and end in the database, and both show the response travelling back to the interface.

Saving a stamp. The collector opens a stamp detail page and presses Save. The client sends POST /api/collection with a JSON body of {"stampId": "..."} and the token in the Authorization header. The authentication middleware verifies the token and attaches the user id. The controller checks that the stamp exists, inserts a collection row and then evaluates the collection achievements. The new state is returned and the button changes to Saved. If the row already exists, the unique index raises a duplicate-key error, which the central handler translates into a conflict rather than a server error, so a double tap does not create a second row.

Creating a post. The collector writes a post, optionally attaches a stamp and submits. The server creates the post, increments the author’s post milestone counters, evaluates the community achievements and creates an achievement notification for each badge that was newly unlocked. Other collectors who follow the author receive a feed entry, and a like or a comment on the post creates a notification for the author, provided the author has left that preference on.

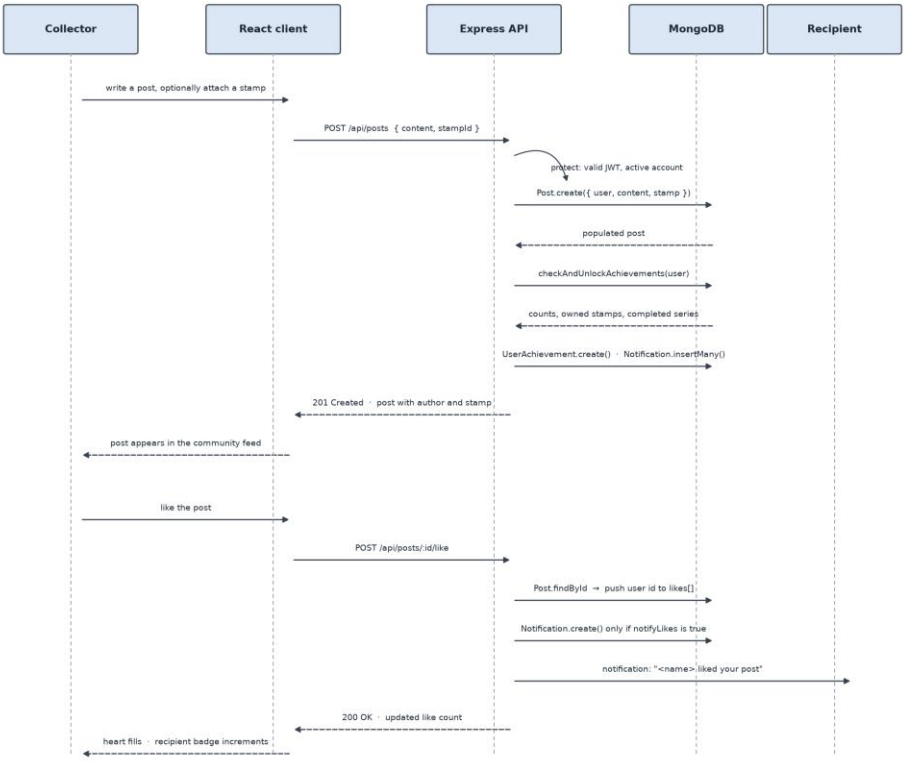


Fig. 4.4 Sequence of creating a post and unlocking an achievement

4.7 API Design

The API is organised around resources and uses the HTTP verbs and status codes in their usual sense. Every protected route reads the token from the Authorization header as a Bearer value. Errors use a single shape, {"message": "..."}, so the client can show the message without guessing which field failed. The table lists the main endpoints; the appendix contains complete request and response examples.

Method and path	Purpose	Access
POST /api/auth/register	Create an account and return a token	Public
POST /api/auth/login	Sign in with an email or username	Public, throttled
GET /api/auth/me	Return the signed-in user	Collector
GET /api/stamps	List, search, filter, sort and paginate stamps	Public
GET /api/stamps/facets	Return countries, categories, themes and years for the filters	Public
GET /api/stamps/:id	Return one stamp	Public
GET /api/countries	List countries with counts	Public

Method and path	Purpose	Access
GET /api/countries/:idOrCode	Return one country and its stamps	Public
POST /api/collection	Add a stamp by id in the body	Collector
DELETE /api/collection/:stampId	Remove a stamp	Collector
GET /api/collection/gaps	Return missing stamps	Collector
GET /api/collection/passport	Return per-country progress	Collector
GET /api/posts	Return the community feed	Public, filtered by privacy
GET /api/posts/saved	Return saved posts	Collector
POST /api/posts	Create a post	Collector
POST /api/posts/:id/like	Toggle a like	Collector
POST /api/posts/:id/save	Toggle a saved post	Collector
POST /api/posts/:id/comments	Add a comment	Collector
GET /api/users/:username	Return a profile	Public or follower
POST /api/users/:id/follow	Toggle a follow	Collector
PUT /api/users/profile	Update the signed-in profile	Collector
GET /api/notifications	Return notifications	Collector
PUT /api/notifications/:id/read	Mark one notification read	Collector
GET /api/admin/stats	Return platform statistics	Admin
PUT /api/admin/users/:id/toggle	Activate or deactivate a user	Admin
POST /api/stamps	Create a stamp	Admin
POST /api/stamps/upload	Store an uploaded stamp image	Admin
DELETE /api/posts/:id	Remove a post	Author or Admin

Catalogue response. The catalogue is the most used endpoint, so its contract is fixed. Filters are optional and combine with AND. Pagination is opt-in: when ?page or ?limit is present the controller returns a page and sets four headers; when neither is present it returns the whole matching array and sets no pagination header, which keeps existing clients compatible. The limit is capped at 60. The example below asks for the second page of six sports stamps from Japan.

```
GET /api/stamps?country=Japan&category=Sports&page=2&limit=6
Authorization: Bearer <token>

HTTP/1.1 200 OK
X-Total-Count: 14
X-Page: 2
```

```
X-Limit: 6
X-Total-Pages: 3
Content-Type: application/json

[
  {
    "_id": "665f1c2a9b1e4a0012ab34cd",
    "name": "Tokyo 1964 - Judo",
    "country": "Japan",
    "year": 1964,
    "category": "Sports",
    "theme": "Sports",
    "denomination": "10 yen",
    "imageUrl": "/api/stamps/files/japan-1964-judo.jpg"
  }
]
```

Error contract. A missing token is 401; a token for the wrong role is 403; a missing record is 404; a duplicate collection row is 409; an invalid category, theme, year or page is 400; an upload of the wrong type is 400 and an oversized upload is 413. The client treats 401 as a signal to clear the session and displays every other message in a toast.

4.8 User Interface Design

The interface uses a warm paper palette with postal red as the primary action colour and deep blue for headings, a deliberate reference to the printed album the platform replaces. Type is set in a serif face for headings and a clean sans-serif for controls, so a catalogue entry reads like a catalogue and a button reads like a control. Every page keeps the same top bar with a search field, a notification bell and the account menu, and on a phone the same destinations move to a compact bottom bar.

The design follows five rules. One primary action per screen: the catalogue saves, the editor publishes, the admin form submits. Progressive disclosure: a stamp card shows the image, the name, the country and the year, and the story appears on the detail page. Visible state: a saved stamp looks different from an unsaved one, and a read notification looks different from an unread one. Empty states teach: a new collection shows the first three stamps to add rather than a blank grid. Accessible contrast and reachable controls: every interactive element is a button or a link with a visible focus ring, and images carry alternative text built from the stamp name and country.

Screen	Main content	Primary action
Landing	Hero, three ways to use the site, featured stamps	Explore the catalogue
Explore	Search, filter panel, stamp grid and pagination	Open a stamp
Stamp detail	Image, story, design note, attribution and related stamps	Save to collection
Countries	Country cards with flags and counts	Open a country

Screen	Main content	Primary action
Timeline	Eras and years in order	Open a stamp
Collection gaps	Missing stamps by country and series	Add a stamp
Stamp passport	Per-country progress and category coverage	Open a country
Community	Feed, composer, likes, comments and saved posts	Create a post
Profile	Collection, posts, followers and interests	Follow or edit
Admin	Statistics, users, stamps and posts	Manage content

4.9 Security Design

Security is designed in layers, so that a mistake in one layer does not expose the system.

- Passwords are hashed with bcrypt at a cost of ten and are selected out of every query by default, so a password hash cannot be returned by accident.
- Authentication uses a signed JWT that expires after thirty days, or after twelve hours when the user does not choose to be remembered. The token carries only the user id and is verified on every protected request.
- Authorisation is checked on the server in three ways: a role guard for admin routes, an ownership check for a resource such as a post or a comment, and a privacy check for a profile, its collection and its posts. Hiding a control in the interface is never treated as protection.
- The login route is throttled to ten attempts per fifteen minutes for each address, which slows down password guessing without locking a legitimate user out for long.
- Uploads are accepted only as PNG, JPEG, WebP or GIF, are checked by their magic bytes rather than their name, and are limited to four megabytes. Stored files are served read-only from a fixed folder, and the client never chooses the path.
- CORS is enabled for the API and must be restricted to the deployed client origin in production; secrets are read from environment variables, and a failed request returns a message rather than a stack trace or a database path.

Deliberate limits. The prototype does not send email, so there is no password reset by mail and no email verification; a forgotten password is reset by an administrator in the demonstration environment. Google sign-in exists as an optional path and is hidden until its client credentials are supplied. The application does not include a web application firewall, a content security policy or rate limiting outside the login route. These are recorded as future work rather than implied to exist.

CHAPTER 5
IMPLEMENTATION

5.1 Implementation Overview

The application is implemented as two npm projects in one repository. The server project contains the Express API, the Mongoose models, the seed and test scripts, and the stamp image folder. The client project contains the React application built with Vite. Both projects are ordinary Node projects with no global tool other than Node.js 18 or later and a running MongoDB server.

Configuration is kept in server/.env, which is not committed. It holds the port, the MongoDB connection string, the token secret and the client URL used by the optional Google callback. A sample file documents the same keys so a new machine can be configured without reading the source. The client reads only the API prefix, which the Vite development server proxies to port 5000; in production the same prefix is served by the reverse proxy in front of the API.

```
server/
  server.js            Express app, routes and error handler
  config/db.js         Mongoose connection
  models/              11 Mongoose schemas
  controllers/         one controller per resource
  routes/              one router per resource
  middleware/          auth.js, rateLimit.js
  utils/               pagination.js, taxonomy.js, achievements.js
  data/                stamps.json and stamp-manifest.json
  public/stamps/       catalogue images served read-only
  seed.js              idempotent catalogue and demo seed
  e2e-test.js          203 end-to-end checks
  svg-render-test.js   checks the generated stamp artwork
client/
  src/pages/           Landing, Explore, StampDetail, Collection,
                       Passport, Community, Profile, Admin, ...
  src/components/      StampCard, PostCard, Navbar, ProtectedRoute
  src/context/         AuthContext
  src/hooks/           usePagedList
  src/services/api.js  the Axios instance
```

Order of work. The catalogue came first, because every other screen depends on it. The taxonomy file fixed the two axes, seven categories and eight themes, and the country and series records were seeded before any stamp. The seed script is idempotent: it upserts countries, series and stamps by their natural key, so it can be run repeatedly during development without duplicating records. Authentication, the collection and the community followed, and the admin console came last, when the three content types it manages already existed.

5.2 Development Phases

Phase	Work carried out	Output

Phase	Work carried out	Output
Requirement analysis	Studied existing catalogues and paper practice; fixed the two-axis taxonomy	Requirement list and data dictionary
Data curation	Selected 59 stamps from 12 countries; recorded story, person and image attribution	stamps.json, stamp-manifest.json
Database and API	Wrote 11 models, controllers, routers, authentication, pagination and error handling	Working REST API
Client	Built the pages, shared components, auth context and the responsive layout	React application
Testing	Seeded a clean database and ran the end-to-end and SVG render suites	203 passing checks
Documentation	Wrote this report and produced the figures, screenshots and appendix	Report and evidence

The phases overlapped in practice. A requirement discovered while building the passport, such as the need to ignore an owned stamp that no longer exists, was fed back into the controller and then into the test suite. The test script grew with the API instead of being written at the end, which is why it covers behaviour that is easy to break silently, such as pagination headers, private-profile filtering and duplicate collection rows.

5.3 Key Functions

5.3.1 Registration and sign-in

Registration validates that all four fields are present, that the password is at least six characters long and that neither the email address nor the username is already in use. The password is hashed with a fresh salt at cost ten; the plain value is never stored or logged. The response uses one shape, publicUser, which contains the fields the client needs and a token, so the collector is signed in immediately after registering.

```
const userExists = await User.findOne({
  $or: [{ email }, { username }],
});
if (userExists) {
  return res.status(400).json({
    message: 'User with this email or username already exists' });
}

const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
const user = await User.create({
  name, username, email, password: hashedPassword,
});
res.status(201).json(publicUser(user));
```

Sign-in accepts an email address or a username, because both columns are unique and a collector should not have to remember which one they used. The identifier is trimmed and lowercased, the account is looked up with an $or, the password is compared

with bcrypt and an inactive account is refused after the password check, so the response does not reveal whether an address exists. A successful login clears the throttle counter for that identity.

5.3.2 Route protection and roles

Every protected route calls protect, which reads the bearer token, verifies its signature and expiry, loads the user and rejects the request if the account has been deactivated since the token was issued. The check happens on the server for every request, so deactivating an account takes effect immediately even though the token itself is still valid. Admin routes add adminOnly after protect, which is why the role is read from the database rather than from the token.

```
const protect = async (req, res, next) => {
  try {
    const token = readToken(req);
    if (!token) {
      return res.status(401).json({ message: 'Not authorized' });
    }
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');
    if (!user || !user.isActive) {
      return res.status(401).json({ message: 'Not authorized' });
    }
    req.user = user;
    next();
  } catch (error) {
    return res.status(401).json({ message: 'Token failed' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') return next();
  return res.status(403).json({ message: 'Not authorized as admin' });
};
```

5.3.3 Catalogue query and pagination

The catalogue controller builds one Mongo query from the request. Free text is escaped and matched case-insensitively across name, person, theme, description, story and country name. A country may be sent as an ObjectId, an ISO code or a name, and is resolved to one id before the query runs. Year filters handle an exact year, a decade and an inclusive range, and the last one applied wins. An invalid category or theme returns 400 with the allowed values instead of quietly returning nothing.

```
const MAX_LIMIT = 60;

const readPaging = (req, defaultLimit = 12) => {
  const rawPage = parseInt(req.query.page, 10);
  const rawLimit = parseInt(req.query.limit, 10);
  const wantsPaging =
    req.query.page !== undefined || req.query.limit !== undefined;
  const page = Number.isNaN(rawPage) || rawPage < 1 ? 1 : rawPage;
  const limit = Number.isNaN(rawLimit)
    ? defaultLimit
    : Math.min(MAX_LIMIT, Math.max(1, rawLimit));
  return { wantsPaging, page, limit, skip: (page - 1) * limit };
};
```

```
const setPagingHeaders = (res, { total, page, limit }) => {
  res.set('X-Total-Count', String(total));
  res.set('X-Page', String(page));
  res.set('X-Limit', String(limit));
  res.set('X-Total-Pages',
    String(Math.max(1, Math.ceil(total / limit))));
};
```

Pagination is deliberately opt-in. The body of the response stays a plain array, and the totals a Load More control needs are sent as headers. When neither parameter is present the endpoint returns the whole matching set, as it did before pagination existed, so no earlier client breaks. The limit is clamped between one and sixty, and an unreadable value falls back to the default, which means a malformed request can slow the server down but cannot ask it for a million rows.

5.3.4 Collection, gaps and passport

Adding a stamp first checks that the stamp id is present, then looks for an existing row and returns a clear message if the stamp is already owned. The insert is still protected by the compound unique index, because two simultaneous requests can both pass the lookup; the index is the rule and the lookup is only a friendly message. After a successful insert the achievement rules run, and the newly created row is returned with its stamp populated so the client can update the card without a second request.

```
const existing = await Collection.findOne({
  user: req.user._id, stamp: stampId,
});
if (existing) {
  return res.status(400).json({
    message: 'Stamp already in collection' });
}

const item = await Collection.create({
  user: req.user._id, stamp: stampId,
});
const populated = await item.populate('stamp');
await checkAndUnlockAchievements(req.user._id);
res.status(201).json(populated);
```

Gaps and the passport are computed, not stored. Both load the collector’s owned ids and the whole catalogue once, then group in memory. Gaps reports only the countries and series that are incomplete, sorted so that the least complete appears first, which turns the page into a to-do list. The passport counts the catalogue of each country, the number owned, the categories covered, the series held and the earliest and latest year owned. Because both views read the live catalogue, adding or deleting a stamp changes them immediately with no rebuild step.

5.3.5 Achievements

Ten rules cover collection size, posts, a historical stamp, three wildlife stamps, five countries and a completed series. Every rule receives the same context object of counts, so a rule cannot bind to the wrong argument. The unlock is written one row at a

time and a duplicate-key error is treated as "already earned" rather than as a failure, which makes the whole operation safe to repeat after every collection change or post.

```
const RULES = [
  { name: 'First Stamp', check: (c) => c.collectionCount >= 1 },
  { name: '10 Stamps', check: (c) => c.collectionCount >= 10 },
  { name: 'First Post', check: (c) => c.postCount >= 1 },
  { name: 'Community Contributor', check: (c) => c.postCount >= 3 },
  { name: 'History Explorer', check: (c) => c.historicalCount >= 1 },
  { name: 'Wildlife Collector', check: (c) => c.wildlifeCount >= 3 },
  { name: 'International Collector',
    check: (c) => c.countryCount >= 5 },
  { name: 'First Series', check: (c) => c.completedSeries >= 1 },
];

try {
  await UserAchievement.create({
    user: userId, achievement: achievement._id,
  });
} catch (error) {
  if (error && error.code === 11000) continue;
  throw error;
}
```

5.3.6 Community privacy

A private profile must be private everywhere, not only on its own page. The feed therefore computes the ids of private collectors and removes their posts unless the viewer is the owner or an approved follower. The same rule is applied when a single post is opened and when comments are listed. The function returns null when nobody is private, which lets the query skip the filter entirely on the common path.

```
const hiddenAuthorsFor = async (viewer) => {
  const privateIds = (await User.find({ isPrivate: true })
    .select('_id')).map((u) => u._id);
  if (!privateIds.length) return null;

  let allowed = [];
  if (viewer) {
    const followed = await Follow.find({
      follower: viewer._id,
      following: { $in: privateIds },
    }).select('following');
    allowed = followed.map((f) => f.following);
  }
  const hidden = privateIds.filter((id) => {
    if (viewer && id.equals(viewer._id)) return false;
    return !allowed.some((a) => a.equals(id));
  });
  return hidden.length ? hidden : null;
};
```

5.3.7 Safe image upload

The admin form sends an image as a base64 data URL, which avoids a multipart parser and keeps the upload on the same JSON path as the rest of the API. The controller decodes the buffer, rejects it above four megabytes, and identifies the format from its leading bytes rather than from the name supplied by the browser. The extension is then chosen from that check, the file name is built from a slug of the stamp name plus a timestamp and a random suffix, and the response returns the read-only URL.

```
const MAX_IMAGE_BYTES = 4 * 1024 * 1024;

const buffer = Buffer.from(match[1], 'base64');
if (buffer.length > MAX_IMAGE_BYTES) {
  return res.status(413).json({
    message: 'Image must be 4MB or smaller' });
}
const kind = EXT_BY_MAGIC.find((k) => k.test(buffer));
if (!kind) {
  return res.status(400).json({
    message: 'Only PNG, JPEG, WebP and GIF are accepted' });
}

fs.writeFileSync(
  path.join(IMAGE_DIR, filename), buffer);
res.status(201).json({
  image: `/api/stamps/files/${filename}`,
  bytes: buffer.length,
});
```

5.4 Deployment

The client is built with npm run build, which produces a folder of static files that any web server can host. The server runs as a long-lived Node process behind a reverse proxy; the proxy serves the static client, forwards /api to the Node process and forwards /api/stamps/files to the same process so the image route keeps working. MongoDB can be a managed cluster, in which case the connection string is the only change required.

Four settings must be supplied in the server environment: the port, the MongoDB URI, a long random JWT secret and the public client URL. If Google sign-in is wanted, its client id, secret and redirect URI are added and the button appears; if they are absent the rest of the application is unaffected. The stamp image folder must be on persistent storage, because a redeploy that replaces the container file system would otherwise drop every uploaded image while the database still holds its URL.

The deployment checklist is short: run the seed once against the production database only if an empty catalogue is intended, confirm that the seed administrator password has been changed, restrict CORS to the deployed client origin, serve the site over HTTPS so the bearer token is not sent in clear text, and confirm that the reverse proxy allows request bodies of at least six megabytes for the image upload. These are deployment steps, not claims that a public instance already exists.

5.5 Summary

The implementation follows the design closely. The API owns every rule, the client owns presentation only, and the derived views are computed from live data so they cannot drift. The code is organised by resource, with one model, controller and router per collection, and the shared behaviour (authentication, pagination, taxonomy, achievements, error handling) lives in one place. The next chapter reports how this implementation was tested.

CHAPTER 6
TESTING

6.1 Testing Approach

Testing is black-box at the HTTP boundary. The suite starts from a freshly seeded database, signs in as the seeded collector and administrator, and then exercises the API through fetch exactly as the browser does. Each check is a single assertion with a readable label, and the script prints PASS or FAIL beside it before a final count. This style was chosen because the same request can be replayed by hand in a terminal when a check fails, which makes a failure easy to reproduce.

Three kinds of test are used. Functional checks confirm that a feature returns the expected data and status. Negative checks confirm that the server refuses what it should refuse: an anonymous request, a non-admin request, an invalid category, a duplicate collection row, a self-follow or a non-image upload. State checks confirm that a write changed exactly what it should and nothing else, for example that removing a stamp reduces the collection count by one and that a duplicate unlock does not create a second badge. A separate render test mounts the stamp artwork component with React's server renderer and inspects the generated SVG.

The seed script is idempotent, so the suite can be run repeatedly. It upserts twelve countries, seven series and fifty-nine stamps, creates the demo accounts and a seeded collection of ten stamps, and then the test creates its own temporary users and content. Temporary rows are removed at the end of the run or left harmless, so a second run starts from the same state as the first.

6.2 Test Environment

Item	Details
Operating system	Windows 11
Runtime	Node.js 18 or later
Database	MongoDB, local instance, freshly seeded
API base URL	http://localhost:5000/api
Client	Vite development server on http://localhost:5173
Browser for manual checks	Google Chrome, responsive mode at 360, 768 and 1366 pixels
Test runner	e2e-test.js and svg-render-test.js, run with npm test and npm run test:svg

Item	Details
Seed data	12 countries, 7 series, 59 stamps, 6 demo accounts, 10 seeded posts

6.3 Test Cases

The table lists representative cases from the suite. Every case in the table was executed and passed; the full suite contains 203 checks across the same areas.

ID	Scenario	Input	Expected result	Status
T01	Sign in with a wrong password	Known email, wrong password	400 with an invalid-credentials message	Pass
T02	Register a duplicate username	Existing username, new email	400, no second account	Pass
T03	Open a protected route anonymously	GET /api/collection without a token	401	Pass
T04	Read the catalogue	GET /api/stamps	All 59 stamps, each with an image path	Pass
T05	Filter by ISO country code	country=KE	Only Kenyan stamps	Pass
T06	Combine filters	country=India & theme=Space	Indian space stamps only	Pass
T07	Page the catalogue	page=2&limit=6	Six rows and correct total headers	Pass
T08	Add the same stamp twice	POST /api/collection twice	First 201, second 400	Pass
T09	Follow yourself	POST /api/users/:id/follow with own id	400, no follow row	Pass
T10	Open a private profile as a stranger	Private account, no follow	canView=false; collection and posts 403	Pass
T11	Open a private profile as a follower	Private account, approved follower	Profile, collection and posts visible	Pass
T12	Delete another collector’s post	Collector token, foreign post id	403	Pass
T13	Deactivate an administrator	Admin token, admin target	400, account unchanged	Pass
T14	Upload a non-image	Text payload as a data URL	400, no file written	Pass
T15	Unlock a badge twice	Two collection writes that satisfy one rule	One userachievement row, one notification	Pass

6.4 Functional Test Results

Area	What was verified	Outcome

Area	What was verified	Outcome
Authentication	Login with email and username, duplicate registration, inactive account, token rejection, password never returned	All checks passed
Catalogue	Full list, detail, search fields, country by name and ISO code, exact year, decade, range, category, theme, series, sorting, facets	All checks passed
Countries and series	Country counts, regions, ISO codes, series counts, series filtering and admin series editing	All checks passed
Collection	Add, duplicate add, remove, restore, gaps totals, passport totals and per-country percentages	All checks passed
Community	Create and delete posts, likes and unlikes, comments, comment ownership, saved posts, feed privacy	All checks passed
Profiles and follows	Profile counts, follow and unfollow, follower lists, self-follow rejection, profile update, interests validation	All checks passed
Settings and privacy	Privacy toggle, notification preferences, password change, private content blocking	All checks passed
Achievements	First Stamp, First Post, History Explorer, locked badges, no duplicates, parallel unlock race	All checks passed
Notifications	Creation, preference suppression, read state, mark all read, unread count	All checks passed
Admin	Statistics, user list, activation toggle, admin protection, stamp create, edit and delete, post deletion, image upload	All checks passed
Artwork	SVG structure, perforation holes, guilloche paths, labels and graceful fallback for unknown values	All checks passed

The complete end-to-end suite reported 203 passed and 0 failed, and the render test reported no failing stamps. The checks that are most valuable are the ones that constrain a whole view rather than a single field: the gaps totals must add up, the passport percentages must equal the rounded ratio of owned to total, and the feed must hide a private author while the profile page is also locked. These invariants catch a class of bug that a per-endpoint test would miss.

6.5 Security Testing

Security was tested by attempting the actions a hostile client would attempt, with the token removed, replaced by a collector token or replaced by an administrator token. The following behaviours were confirmed:

- An anonymous request to a protected route returns 401, and a collector token on an admin route returns 403, on every admin endpoint tested.
- No response contains a password field, including the admin user list and the public profile of another collector.

- A deactivated account cannot sign in, and an existing token stops working because the middleware reloads the user and checks isActive.
- A private profile hides its collection and its posts from a non-follower with 403, while an approved follower receives the same content as the owner.
- A collector cannot delete another collector's post or comment, and cannot follow themselves.
- An administrator cannot deactivate an administrator, so the platform cannot lose its last administrator by accident.
- An upload whose payload is not a real image is rejected before any file is written, and the four-megabyte limit is enforced on the decoded bytes.

6.6 Defects Found and Corrected

The suite and manual review found several defects that were fixed before the final run. They are recorded here because each one changed the design, not only a line of code.

- Pagination broke existing clients when it was always on. It was made opt-in: the body stays an array and the totals are sent as headers only when a caller asks for a page.
- A duplicate collection request could pass the lookup and then hit the unique index. The index is now treated as the rule and the duplicate-key error is translated into a clear conflict.
- Two simultaneous achievement checks could both try to insert the same badge. The insert is now handled row by row, and a duplicate key is treated as already earned instead of aborting the remaining unlocks.
- The community feed showed posts from private profiles that the profile page had already hidden. The feed now applies the same privacy filter, including for comments and single-post reads.
- Deleting a stamp left collection rows behind, which inflated the owned count. Deletion now removes those rows, and the derived views also ignore an id that no longer resolves to a stamp.
- A settings path was being captured as a username because it was declared after the dynamic profile route. Static routes are now declared first.

The final run used the corrected code, a freshly seeded database and a production build of the client. No check failed, and no warning about an unhandled promise rejection appeared in the server log during the run.

CHAPTER 7
CONCLUSION

7.1 Summary of the Work

This project set out to make stamp collecting searchable, recordable and social, and it delivered a working full stack application that does those three things. National Philately holds a curated catalogue of fifty-nine stamps from twelve countries, classified on two independent axes and linked to seven series. A collector can search and filter that catalogue, record the stamps they own, see what is missing, measure progress per country, share a find, follow other collectors and earn badges for milestones. An administrator can manage the content behind all of it.

The implementation follows the design in Chapter 4. React and Vite form the presentation tier, Express and Mongoose form the application tier, and MongoDB forms the data tier. Eleven collections carry the data, with unique indexes on the three relationships that must not duplicate: a collection entry, a follow and an unlocked badge. Authentication is token-based, authorisation is checked on the server for every protected request, and the derived views are computed from live data rather than stored.

7.2 Outcomes

- A browsable catalogue with free-text search, combined filters, four sort orders, country pages and a timeline.
- A personal collection with Collection Gaps and a Stamp Passport that answers what to collect next.
- A community with posts, optional stamp attachments, likes, comments, saved posts, following and a private-profile option applied consistently across the site.
- Ten achievement rules and a notification system that respects the collector’s preferences.
- An admin console for statistics, accounts, stamps, images and post moderation, with cascade deletion that keeps derived counts honest.
- A test suite of 203 checks and a separate artwork render test, both passing on a clean seed.

The most useful outcome is not a single screen but the consistency between screens. The same privacy rule governs the profile, the collection, the feed and the comments; the same catalogue drives the gaps, the passport and the country pages; and

the same pagination contract serves the catalogue and the feed. That consistency is what makes the application feel like one product rather than a set of demonstrations.

7.3 Limitations

The prototype has clear limits. The catalogue is curated by hand, so it is small and its growth depends on manual data entry. There is no price or valuation data, because a trustworthy price source was outside the scope. There is no image recognition: a collector cannot photograph a stamp and be told what it is. Email is not configured, so password reset and address verification are administrative actions. Google sign-in is implemented but optional and hidden until credentials are supplied. The rate limiter keeps its counters in process memory, which is correct for one server but not for a cluster. Finally, the deployment steps are documented, but no public instance is claimed.

7.4 Future Enhancements

- A want list and a trade suggestion that compares two collectors' gaps and proposes a mutually useful exchange, without handling money.
- Import and export in a standard exchange format.
- A mobile application that reuses the same API, with offline reading of a downloaded catalogue.
- Image recognition to suggest a stamp from a photograph, with the catalogue entry confirmed by the collector rather than assumed.
- A public API with documented rate limits, so clubs and researchers can build on the catalogue.
- Stronger production security: a content security policy, a distributed rate limiter, email verification and audited administrator actions.
- Multilingual catalogue text and a second currency display, which would make the catalogue useful outside the current audience.

7.5 Conclusion

National Philately shows that a familiar hobby can be improved by a well-modelled database and a careful web application. The catalogue gives stamps a shared description, the collection gives the collector a record, the gaps and passport give the hobby a sense of direction, and the community gives it a place to talk. The hardest parts were not the screens but the invariants behind them: a stamp owned once, a follow in one direction, a private profile private everywhere and a badge earned once. Those invariants now hold, and the test suite proves it.

APPENDIX A
REPRESENTATIVE SOURCE CODE

The code below is taken from the working project and is reproduced with the comments that explain each decision. Long files are excerpted; an ellipsis marks a removed section. The complete source is in the repository folder described in Section 5.1.

A.1 Express application bootstrap

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();
app.use(cors());
app.use(express.json({ limit: '6mb' }));
connectDB();

app.use('/api/stamps/files',
  express.static(path.join(__dirname, 'public', 'stamps'),
    { dotfiles: 'deny', index: false }));
app.use('/api/auth', authRoutes);
app.use('/api/stamps', stampRoutes);
app.use('/api/collection', collectionRoutes);
app.use('/api/posts', postRoutes);
app.use('/api', (req, res) => res.status(404).json({
  message: `API route not found: ${req.method} ${req.originalUrl}` }));
```

A.2 User schema

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  username: { type: String, required: true, unique: true,
              trim: true, lowercase: true },
  email: { type: String, required: true, unique: true,
           trim: true, lowercase: true },
  password: { type: String, required: true },
  profileImage: { type: String, default: '' },
  bio: { type: String, default: '' },
  interests: [{ type: String, trim: true }],
  isPrivate: { type: Boolean, default: false },
  notifyLikes: { type: Boolean, default: true },
  notifyComments: { type: Boolean, default: true },
  notifyFollows: { type: Boolean, default: true },
  notifyAchievements: { type: Boolean, default: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  isActive: { type: Boolean, default: true },
  googleId: { type: String, default: undefined,
              sparse: true, unique: true },
}, { timestamps: true });
```

A.3 Stamp schema and indexes

```
const stampSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  category: { type: String, required: true, trim: true },
  theme: { type: String, required: true, trim: true },
  country: { type: mongoose.Schema.Types.ObjectId,
             ref: 'Country', required: true },
  countryName: { type: String, required: true, trim: true },
  year: { type: Number, required: true },
  series: { type: mongoose.Schema.Types.ObjectId,
            ref: 'Series', default: null },
  seriesNumber: { type: Number, default: null },
  denomination: { type: String, default: '' },
  person: { type: String, default: '' },
  story: { type: String, default: '' },
```

```
  image: { type: String, default: '' },
  sourceUrl: { type: String, default: '' },
  credit: { type: String, default: '' },
  license: { type: String, default: '' },
  viewCount: { type: Number, default: 0 },
}, { timestamps: true });

stampSchema.index({ category: 1, theme: 1 });
stampSchema.index({ country: 1, year: 1 });
stampSchema.index({ year: 1 });
stampSchema.index({ series: 1, seriesNumber: 1 });
stampSchema.index({ viewCount: -1 });
```

A.4 Collection schema with its unique index

```
const collectionSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId,
          ref: 'User', required: true },
  stamp: { type: mongoose.Schema.Types.ObjectId,
           ref: 'Stamp', required: true },
  addedAt: { type: Date, default: Date.now },
}, { timestamps: true });

// A stamp can appear at most once in a collector's collection.
collectionSchema.index({ user: 1, stamp: 1 }, { unique: true });
```

A.5 Follow schema and self-follow guard

```
const followSchema = new mongoose.Schema({
  follower: { type: mongoose.Schema.Types.ObjectId,
              ref: 'User', required: true, index: true },
  following: { type: mongoose.Schema.Types.ObjectId,
               ref: 'User', required: true, index: true },
}, { timestamps: true });

followSchema.index({ follower: 1, following: 1 }, { unique: true });

followSchema.pre('validate', function preventSelfFollow(next) {
  if (this.follower && this.following &&
      this.follower.equals(this.following)) {
    return next(new Error('You cannot follow yourself'));
  }
  next();
});
```

A.6 Authentication and role middleware

```
const protect = async (req, res, next) => {
  try {
    const token = readToken(req);
    if (!token) {
      return res.status(401).json({ message: 'Not authorized, no token' });
    }
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');
    if (!user) {
      return res.status(401).json({ message: 'User not found' });
    }
    if (!user.isActive) {
      return res.status(401).json({ message: 'Account deactivated' });
    }
    req.user = user;
    next();
  } catch (error) {
    return res.status(401).json({ message: 'Token failed' });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') return next();
  return res.status(403).json({ message: 'Not authorized as admin' });
};
```

A.7 Registration

```
if (!name || !username || !email || !password) {
  return res.status(400).json({ message: 'Please provide all fields' });
}
if (password.length < 6) {
  return res.status(400).json({
    message: 'Password must be at least 6 characters' });
}
const userExists = await User.findOne({
  $or: [{ email }, { username }],
});
if (userExists) {
  return res.status(400).json({
    message: 'User with this email or username already exists' });
}
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
const user = await User.create({
  name, username, email, password: hashedPassword,
});
res.status(201).json(publicUser(user));
```

A.8 Login with either identifier

```
const { email, username, password, remember = true } = req.body;
const identifier = String(email || username || '')
  .trim().toLowerCase();

const user = await User.findOne({
  $or: [{ email: identifier }, { username: identifier }],
});
if (!user) {
  return res.status(400).json({ message: 'Invalid email or password' });
}
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) {
  return res.status(400).json({ message: 'Invalid email or password' });
}
if (!user.isActive) {
  return res.status(400).json({ message: 'Account has been deactivated' });
}
resetLimit(req);
res.json({ ...publicUser(user),
           token: generateToken(user._id, remember) });
```

A.9 Catalogue query builder

```
const escapeRegex = (s) =>
  String(s).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

if (q.search) {
  const rx = new RegExp(escapeRegex(q.search.trim()), 'i');
  query.$or = [
    { name: rx }, { person: rx }, { theme: rx },
    { description: rx }, { story: rx }, { countryName: rx },
  ];
}
if (q.country) {
  query.country = await buildCountryFilter(q.country);
}
if (q.year !== undefined && q.year !== '') {
  const y = parseInt(q.year, 10);
  if (!Number.isNaN(y)) years.$eq = y;
}
if (q.decade) {
  const d = parseInt(q.decade, 10);
  if (!Number.isNaN(d)) { years.$gte = d; years.$lte = d + 9; }
}
```

A.10 Optional pagination utility

```
const MAX_LIMIT = 60;
```

```
const readPaging = (req, defaultLimit = 12) => {
  const rawPage = parseInt(req.query.page, 10);
  const rawLimit = parseInt(req.query.limit, 10);
  const wantsPaging =
    req.query.page !== undefined || req.query.limit !== undefined;
  const page = Number.isNaN(rawPage) || rawPage < 1 ? 1 : rawPage;
  const limit = Number.isNaN(rawLimit)
    ? defaultLimit
    : Math.min(MAX_LIMIT, Math.max(1, rawLimit));
  return { wantsPaging, page, limit, skip: (page - 1) * limit };
};

const setPagingHeaders = (res, { total, page, limit }) => {
  res.set('X-Total-Count', String(total));
  res.set('X-Page', String(page));
  res.set('X-Limit', String(limit));
  res.set('X-Total-Pages',
    String(Math.max(1, Math.ceil(total / limit))));
};
```

A.11 Adding and removing a collection entry

```
const { stampId } = req.body;
if (!stampId) {
  return res.status(400).json({ message: 'Stamp ID is required' });
}
const existing = await Collection.findOne({
  user: req.user._id, stamp: stampId,
});
if (existing) {
  return res.status(400).json({ message: 'Stamp already in collection' });
}
const item = await Collection.create({
  user: req.user._id, stamp: stampId,
});
const populated = await item.populate('stamp');
await checkAndUnlockAchievements(req.user._id);
res.status(201).json(populated);

const remove = await Collection.findOneAndDelete({
  user: req.user._id, stamp: req.params.stampId });
```

A.12 Collection Gaps grouping

```
const { ownedIds, stamps } = await loadCatalogueState(req.user._id);
const byCountry = new Map();

for (const stamp of stamps) {
  const owned = ownedIds.has(String(stamp._id));
  const country = stamp.country;
  if (!country) continue;
  const key = String(country._id);
  if (!byCountry.has(key)) {
    byCountry.set(key, {
      country: { _id: country._id, name: country.name,
                 iso2: country.iso2, flag: country.flag },
      total: 0, owned: 0, missing: [],
    });
  }
  const c = byCountry.get(key);
  c.total += 1;
  if (owned) c.owned += 1;
  else c.missing.push({ _id: stamp._id, name: stamp.name,
                        year: stamp.year, category: stamp.category });
}
```

A.13 Stamp Passport calculation

```
const pages = countries.map((country) => {
  const own = stamps.filter((s) =>
    s.country && String(s.country._id) === String(country._id));
  const ownedStamps = own.filter((s) =>
    ownedIds.has(String(s._id)));
  const categoryCounts = Object.fromEntries(
    CATEGORIES.map((c) => [c.name, 0]));
```

```
  let earliest = null, latest = null;

  for (const s of own) {
    categoryCounts[s.category] += 1;
    if (ownedIds.has(String(s._id))) {
      if (!earliest || s.year < earliest) earliest = s.year;
      if (!latest || s.year > latest) latest = s.year;
    }
  }
  return { country, total: own.length, owned: ownedStamps.length,
           percent: pct(ownedStamps.length, own.length),
           earliest, latest };
});
```

A.14 Achievement rules and idempotent unlock

```
const RULES = [
  { name: 'First Stamp', check: (c) => c.collectionCount >= 1 },
  { name: 'First Post', check: (c) => c.postCount >= 1 },
  { name: '10 Stamps', check: (c) => c.collectionCount >= 10 },
  { name: '50 Stamps', check: (c) => c.collectionCount >= 50 },
  { name: '100 Stamps', check: (c) => c.collectionCount >= 100 },
  { name: 'Community Contributor', check: (c) => c.postCount >= 3 },
  { name: 'History Explorer', check: (c) => c.historicalCount >= 1 },
  { name: 'Wildlife Collector', check: (c) => c.wildlifeCount >= 3 },
  { name: 'International Collector', check: (c) => c.countryCount >= 5 },
  { name: 'First Series', check: (c) => c.completedSeries >= 1 },
];

try {
  await UserAchievement.create({
    user: userId, achievement: achievement._id });
} catch (error) {
  if (error && error.code === 11000) continue;
  throw error;
}
```

A.15 Privacy filter for the community feed

```
const hiddenAuthorsFor = async (viewer) => {
  const privateUsers = await User.find({ isPrivate: true }).select('_id');
  const privateIds = privateUsers.map((u) => u._id);
  if (!privateIds.length) return null;

  let allowed = [];
  if (viewer) {
    const followed = await Follow.find({
      follower: viewer._id,
      following: { $in: privateIds },
    }).select('following');
    allowed = followed.map((f) => f.following);
  }
  const hidden = privateIds.filter((id) => {
    if (viewer && id.equals(viewer._id)) return false;
    return !allowed.some((a) => a.equals(id));
  });
  return hidden.length ? hidden : null;
};
```

A.16 Admin account toggle and cascade delegation

```
const user = await User.findById(req.params.id);
if (!user) {
  return res.status(404).json({ message: 'User not found' });
}
if (user.role === 'admin') {
  return res.status(400).json({ message: 'Cannot deactivate admin' });
}
user.isActive = !user.isActive;
await user.save();
res.json({ message: `User ${user.isActive ? 'activated' : 'deactivated'}`,
           isActive: user.isActive });

// Admin deletion delegates to the one cascade handler.
```

```
const deleteStamp = async (req, res) => deleteStampCascade(req, res);
const deletePost = async (req, res) => deletePostCascade(req, res);
```

A.17 Image upload validation

```
const match = /^data:image\/(?:png|jpeg|jpg|webp|gif); base64,(.+)$/
  .exec(String(image || ''));
if (!match) {
  return res.status(400).json({
    message: 'Please provide an image as a base64 data URL' });
}
const buffer = Buffer.from(match[1], 'base64');
if (buffer.length > 4 * 1024 * 1024) {
  return res.status(413).json({ message: 'Image must be 4MB or smaller' });
}
const kind = EXT_BY_MAGIC.find((k) => k.test(buffer));
if (!kind) {
  return res.status(400).json({
    message: 'Only PNG, JPEG, WebP and GIF images are accepted' });
}
fs.writeFileSync(path.join(IMAGE_DIR, filename), buffer);
res.status(201).json({ image: `/api/stamps/files/${filename}` });
```

A.18 Sample API exchange

```
POST /api/collection
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{ "stampId": "665f1c2a9b1e4a0012ab34cd" }

HTTP/1.1 201 Created
Content-Type: application/json

{
  "_id": "6660a1f39b1e4a0012ab3510",
  "user": "665f0e1a9b1e4a0012ab3401",
  "stamp": {
    "_id": "665f1c2a9b1e4a0012ab34cd",
    "name": "Tokyo 1964 - Judo",
    "countryName": "Japan",
    "year": 1964
  },
  "addedAt": "2026-10-22T09:14:07.212Z"
}
```

SCREENSHOTS OF THE PROJECT

The screens below are captured from the running application at a desktop width. They follow the order a new collector would use: arrive, browse, open a stamp, review the collection, then take part in the community and, for an administrator, manage the catalogue. The mobile layout keeps the same actions in a compact bottom bar.

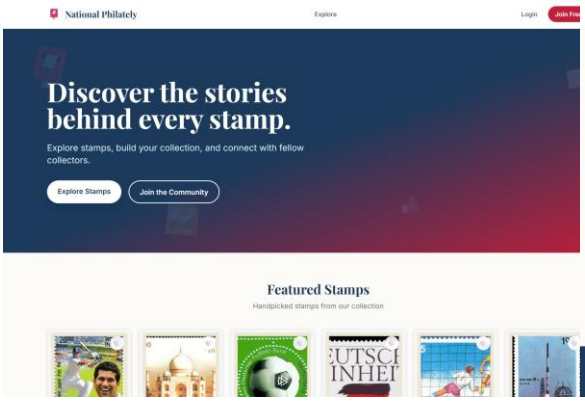


Fig. 7.1 Landing page

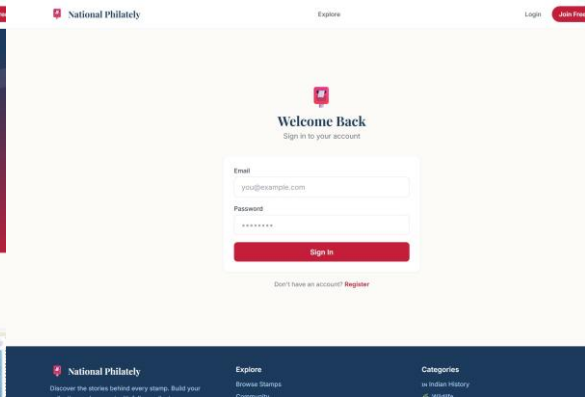


Fig. 7.2 Sign-in page



Fig. 7.3 Explore with filters



Fig. 7.4 Stamp detail

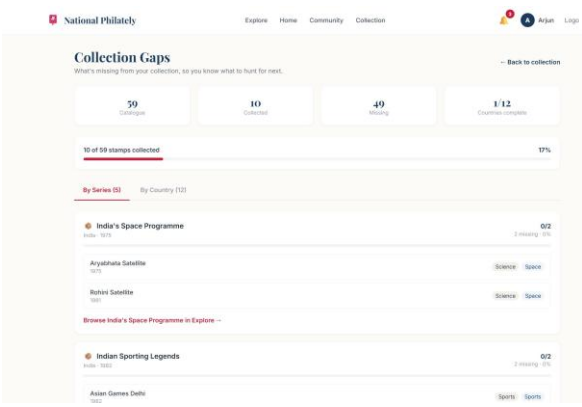


Fig. 7.5 Collection Gaps

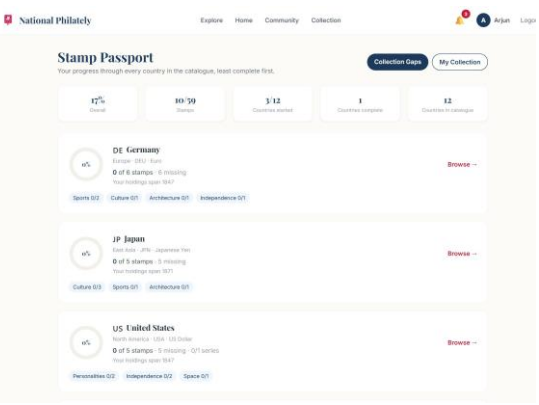


Fig. 7.6 Stamp Passport

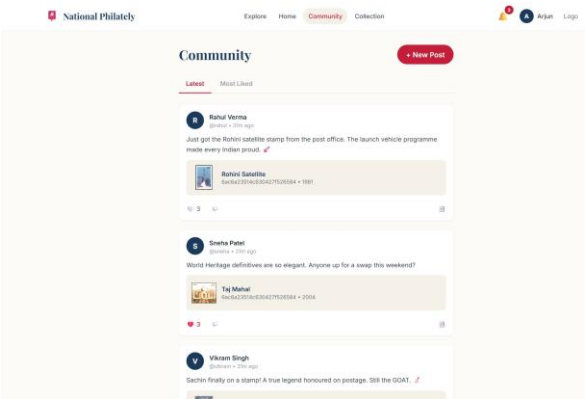


Fig. 7.7 Community feed

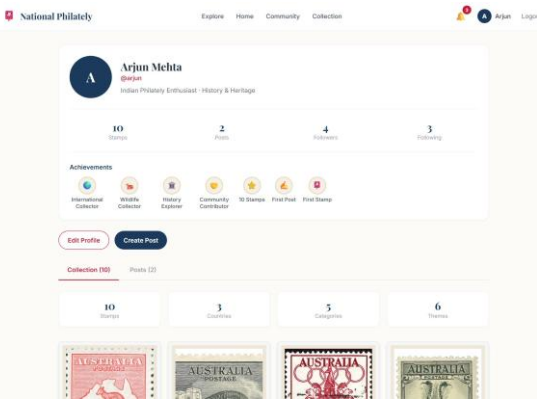


Fig. 7.8 Collector profile



Fig. 7.9 Admin stamp management

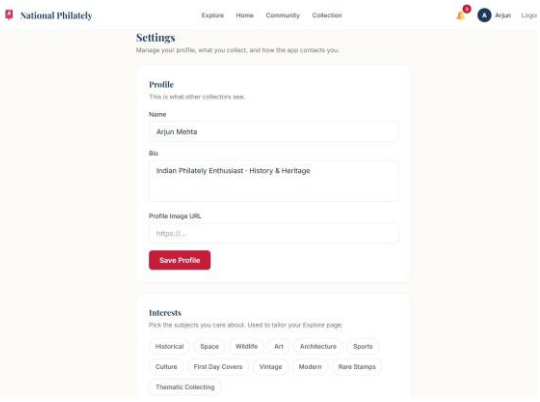


Fig. 7.10 Account settings

The landing page names the three actions the platform supports, Discover, Collect and Connect, and shows a small selection of the catalogue before a visitor commits to an account. The explore page keeps the search field, the filter panel and the result count in one view, so a filter can be added or removed without losing the result. The stamp detail page carries the story, the design note and the full image attribution. Collection Gaps and the Stamp Passport turn the same catalogue into a plan, and the community, profile and admin screens show the three kinds of content the site manages.

REFERENCES

[1] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. https://www.rfc-editor.org/rfc/rfc7519

[2] OWASP Foundation, "OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks," 2021. https://owasp.org/Top10/

[3] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," Proceedings of the USENIX Annual Technical Conference, Monterey, 1999.

[4] MongoDB Inc., "MongoDB Manual: Indexes," MongoDB Documentation. https://www.mongodb.com/docs/manual/indexes/

[5] Mongoose, "Mongoose Documentation: Schemas and Models." https://mongoosejs.com/docs/

[6] OpenJS Foundation, "Express 4.x API Reference." https://expressjs.com/en/4x/api.html

[7] Meta Open Source, "React Documentation: Describing the UI." https://react.dev/learn

[8] Vite, "Vite Guide: Getting Started." https://vitejs.dev/guide/

[9] Tailwind Labs, "Tailwind CSS Documentation." https://tailwindcss.com/docs

[10] Wikimedia Foundation, "Reusing Content Outside Wikimedia," Wikimedia Commons. https://commons.wikimedia.org/wiki/Commons:Reusing_content_outside_Wikimedia

[11] Smithsonian Institution, "National Postal Museum: Collections," Washington, DC. https://postalmuseum.si.edu/

[12] Colnect, "Stamp Catalog." https://colnect.com/en/stamps

[13] StampWorld, "The World’s Largest Stamp Catalogue." https://www.stampworld.com/

[14] International Organization for Standardization, "ISO 3166 Country Codes." https://www.iso.org/iso-3166-country-codes.html